

编译原理第十一次作业参考答案

练习 7.1.1

考虑如下 C 程序，取实参 $a = 2$ ，讨论 $h(2)$ 调用 k 、且 k 即将返回时刻的运行时栈：

```
int k(int *, int);
int h(int a) {
    int i = a * 3 + 1;
    return k(&i, a);
}
int k(int *p, int q) {
    int s = *p + q;
    return s + 2;
}
```

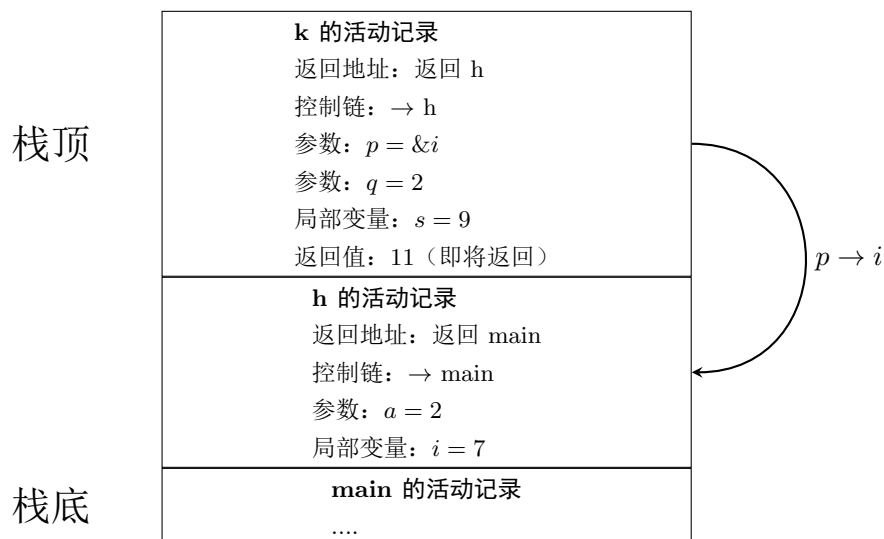
先根据 $a=2$ 把各值算出来：

$h(2) : i = a \times 3 + 1 = 2 \times 3 + 1 = 7$, 随后调用 $k(\&i, 2)$.

$k : p = \&i, q = 2, s = *p + q = 7 + 2 = 9$, 返回 $s + 2 = 11$.

实参 p 的值是 $\&i$ ，它指向 h 帧中的局部变量 i ；因此 k 执行期间 h 帧必须仍然有效。

运行时栈示意。 此刻栈上有两帧： h （较早，在下）与 k （栈顶，在上）。每帧内部按图 7-7 约定，实参与返回值置于帧底（靠近主调者），局部数据置于帧顶。



下表对栈中每个元素给出： 所属活动记录、由谁创建空间、由谁写入、以及此刻的值。

元素	所属活动记录	创建空间	写入者	值
参数 a	h	main	main	2
返回值	h	main	h (k 返回后；待写入)	→ 11
控制链	h	调用序列	main	→ main 帧
局部 i	h	h	h	7
参数 p	k	h	h	$\&i$ (→ $h.i$)
参数 q	k	h	h	2
返回值	k	调用序列（帧开头）	k	11
控制链	k	调用序列	h	→ h 帧
局部 s	k	k	k	9

- 控制链：记录调用者和被调用者的关系， $k \rightarrow h \rightarrow \text{main}$
- 参数 $p = \&i$ 指向 h 帧里的局部 i ，故 k 运行期间 h 帧必须仍然有效
- 此刻 k 即将返回： k 已写好自身返回值 11，而 h 的返回值尚未写入——要等 k 返回后再由 h 写。

- C 语言不支持函数嵌套，故不需要访问链。

练习 7.1.2

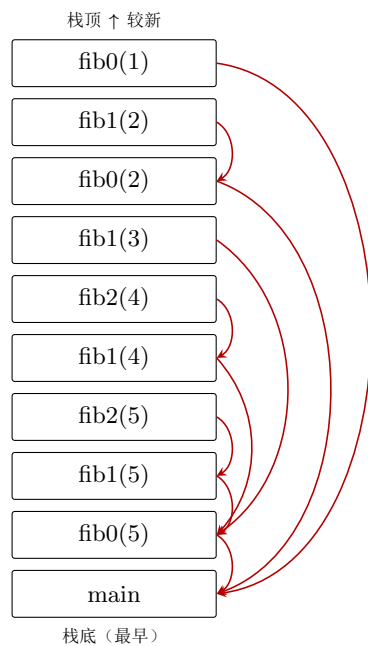
考虑如下嵌套函数程序，词法嵌套层次为 $\text{main} \supset \text{fib0} \supset \text{fib1} \supset \text{fib2}$ (深度 1 ~ 4)。从对 main 的调用开始，追踪到第一次 $\text{fib0}(1)$ 即将返回的时刻：

```
fun main() {
  let fun fib0(n) =
    let fun fib1(n) =
      let fun fib2(n) = fib1(n-1) + fib1(n-2)
      in if n >= 4 then fib2(n)
         else fib0(n-1) + fib0(n-2)
    in if n >= 2 then fib1(n) else n end
  in fib0(5) end;
}
```

调用链（约定总是先求左操作数）。

```
main    → fib0(5)
fib0(5)  n=5 ≥ 2 → fib1(5)
fib1(5)  n=5 ≥ 4 → fib2(5)
fib2(5)  → fib1(4) [左 : fib1(n-1)]
fib1(4)  n=4 ≥ 4 → fib2(4)
fib2(4)  → fib1(3) [左 : fib1(n-1)]
fib1(3)  n=3 < 4 → fib0(2) [左 : fib0(n-1)]
fib0(2)  n=2 ≥ 2 → fib1(2)
fib1(2)  n=2 < 4 → fib0(1) [左 : fib0(n-1)]
fib0(1)  n=1 < 2 → 返回 n = 1 ★
```

(1) 活动记录栈与访问链。 访问链指向最近活动记录，下图自底（main，最早）向上（ $\text{fib0}(1)$ ，栈顶）排列，右侧弧线为各帧的访问链。



(2) **display** 实现：当前 $d[1..4]$ 与各帧保存的旧条目。 进入深度为 i 的函数时，先保存当前 $d[i]$ （存入本帧），再令 $d[i]$ 指向本帧；该函数只改动 $d[i]$ 一个表项。按调用顺序模拟到 $\text{fib0}(1)$ 即将返回，得当前 **display**：

条目	指向的活动记录
$d[1]$	main
$d[2]$	fib0(1) ← 当前帧（深度 2）
$d[3]$	fib1(2) （更深调用留下的遗留值）
$d[4]$	fib2(4) （更深调用留下的遗留值）

当前在 $\text{fib0}(1)$ （深度 2）中执行，访问非局部变量只用 $d[1]$ 、 $d[2]$ ； $d[3]$ 、 $d[4]$ 是更深调用遗留的旧值，进入深度 2 的函数时不会改动它们，故它们仍有值、并非为空。

各帧进入时所保存的 **display** 旧条目（仅保存本帧深度对应的那一项）：

活动记录	深度	进入时保存的 $d[i]$ 旧值
main	1	$d[1] \leftarrow \text{空}$
fib0(5)	2	$d[2] \leftarrow \text{空}$
fib1(5)	3	$d[3] \leftarrow \text{空}$
fib2(5)	4	$d[4] \leftarrow \text{空}$
fib1(4)	3	$d[3] \leftarrow \text{fib1}(5)$
fib2(4)	4	$d[4] \leftarrow \text{fib2}(5)$
fib1(3)	3	$d[3] \leftarrow \text{fib1}(4)$
fib0(2)	2	$d[2] \leftarrow \text{fib0}(5)$
fib1(2)	3	$d[3] \leftarrow \text{fib1}(3)$
fib0(1)	2	$d[2] \leftarrow \text{fib0}(2)$